

1 /*

2
3 Deque interface:

4
5 --> The Deque interface of the Java collections framework provides the
functionality of a double-ended queue.

6 --> It extends the Queue interface.

7 --> In a regular queue, elements are added from the rear and removed
from the front.

8 --> However, in a deque, we can insert and remove elements from both
front and rear.

9 --> In order to use the functionalities of the Deque interface, we need
to use classes that implement it:

10 1. LinkedList

11 2. ArrayDeque

12
13 --> Since Deque extends the Queue interface, it inherits all the methods
of the Queue interface.

14
15 Besides methods available in the Queue interface, the Deque interface
also includes the following methods:

16
17 1. void addFirst() - Adds the specified element at the beginning of the
deque. Throws an exception if the deque is full.

18 2. void addLast() - Adds the specified element at the end of the deque.
Throws an exception if the deque is full.

19 3. boolean offerFirst() - Adds the specified element at the beginning
of the deque. Returns false if the deque is full.

20 4. boolean offerLast() - Adds the specified element at the end of the
deque. Returns false if the deque is full.

21 5. Object getFirst() - Returns the first element of the deque. Throws
an exception if the deque is empty.

22 6. Object getLast() - Returns the last element of the deque. Throws an
exception if the deque is empty.

23 7. Object peekFirst() - Returns the first element of the deque. Returns
null if the deque is empty.

24 8. Object peekLast() - Returns the last element of the deque. Returns
null if the deque is empty.

25 9. Object removeFirst() - Returns and removes the first element of the
deque. Throws an exception if the deque is empty.

26 10. Object removeLast() - Returns and removes the last element of the
deque. Throws an exception if the deque is empty.

27 11. Object pollFirst() - Returns and removes the first element of the
deque. Returns null if the deque is empty.

28 12. Object pollLast() - Returns and removes the last element of the
deque. Returns null if the deque is empty.

29 13. boolean add() - Adds elements

30 14. boolean addAll() - Adds all the elements of given objects

31 15. void clear() - removes all the elements

32 16. boolean isEmpty() - returns boolean if given object is empty

33 17. Object remove() - removes first element from given object

34 18. boolean remove(Object o) - removes specific element from given
object, if given element not available then returns false.

35 18. int size() - gives count of total number of elements present in the
object

36
37 --> The Stack class of the Java Collections framework provides the
implementation of the stack.
38 --> However, it is recommended to use Deque as a stack instead of the
Stack class.
39 --> It is because methods of Stack are synchronized.

40
41 Here are the methods the Deque interface provides to implement stack:

- 42
43 1. push() - adds an element at the beginning of deque
44 2. pop() - removes an element from the beginning of deque
45 3. peek() - returns an element from the beginning of deque
46

47 */

48
49 import java.util.ArrayDeque;
50 import java.util.ArrayList;
51 import java.util.Deque;
52
53 public class Collection8 {
54
55 public static void main(String[] args) {
56
57 ArrayList<Integer> al = new ArrayList<>();
58 al.add(30);
59 al.add(20);
60 al.add(10);
61 al.add(40);
62 System.out.println("ArrayList is: "+al);
63
64 ArrayDeque<Integer> dq = new ArrayDeque<>();
65 ArrayDeque dq1 = new ArrayDeque<>(10);
66 ArrayDeque dq2 = new ArrayDeque<>(al);
67
68 System.out.println("dq is: "+dq);
69 System.out.println("dq1 is: "+dq1);
70 System.out.println("dq2 is: " + dq2);
71
72 dq1.add(12);
73 dq1.add("Hi");
74
75 dq.addAll(dq2);
76
77
78
79 dq1.offer(13);
80 dq1.offer("Hey");
81 dq1.offerFirst(10); // NullPointerException if element passed is
"null"
82 dq1.offerLast(20.32); // NullPointerException if element passed
is "null"
83
84 dq.addFirst(9); // NullPointerException if element passed is
"null"
85 dq.addLast(16); // NullPointerException if element passed is

```
86         "null"
87
88
89     System.out.println(dq1);
90
91     System.out.println(dq1.getFirst()); // NoSuchElementException if
    empty
92     System.out.println(dq1.getLast()); // NoSuchElementException if
    empty
93
94     System.out.println(dq1.peek()); // peek returns null if empty
95
96     System.out.println(dq1.peekFirst()); // peekFirst returns null
    if empty
97     System.out.println(dq1.peekLast()); // peekLast returns null if
    empty
98
99     System.out.println(dq1.remove());
100    System.out.println(dq1.remove(10));
101
102    System.out.println(dq1.removeFirst()); // NoSuchElementException
    if empty
103    System.out.println(dq1.removeLast()); // NoSuchElementException
    if empty
104
105    System.out.println(dq1.poll()); // poll returns null if empty
106    System.out.println(dq1.pollFirst()); // pollFirst returns null
    if empty
107    System.out.println(dq1.pollLast()); // pollLast returns null if
    empty
108
109    dq.clear();
110
111    System.out.println(dq1.size());
112
113    }
114
115 }
```